

How to write an amazing article

Adam Herout*

Abstract

What is the problem? What is the topic? The aim of this paper? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. How is the problem solved, the aim achieved (methodology)? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. Curabitur massa neque, commodo posuere fringilla ut, cursus at dui. Nulla quis purus a justo pellentesque. What are the specific results? How well is the problem solved? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. So what? How useful is this to Science and to the reader? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod.

Keywords: Keyword1 — Keyword2 — Keyword3

Supplementary Material: [Demonstation video](#) — [Downloadable code](#)

*herout@fit.vut.cz, Faculty of Information Technology, Brno University of Technology

1. Introduction

With the rise of Large Language Models (LLMs) and the increase in their capabilities, companies are motivated to use LLMs for automation tasks, including testing and the tooling for it. However, LLMs are not plug-and-play; specialized tooling is often required to produce reliable outputs. This work defines a formal base into which LLMs rewrite requirements and test cases written in natural language, so that the result is parsable by test coverage checking systems.

There is an increasing demand to instrument systems in natural language, including the specification of requirements and their test cases for testing systems and system components. Even when requirements and test cases are written in English, they must still be rewritten into a mathematical formal base so that existing test-coverage techniques can be applied. A formal base into which LLMs can convert human-written text is therefore needed. This formal base must be able to express all motivational requirement examples. Its semantics must be easily interpretable by LLMs and

agentic systems, so that agents can rewrite requirements and test case descriptions from human language into the formal base with high accuracy and without changing the semantics; the formalism itself must be unambiguous. It must also be convertible to existing formal bases used by formal verification tools. A good formalism should meet all of the mentioned criteria.

2. Existing methods of requirement formalisation – State of the Art

The translation from natural-language (NL) requirements to formal specifications has long been pursued through controlled-language frontends that constrain the input enough to make it unambiguous and further processable. The recent maturation of large language models has reopened the question of to what degree this translation can be automated without such restrictions, leading to LLM-based pipelines and hybrid approaches that pair LLMs with formal provers and solvers — the rest of this section surveys that landscape. This overview is informed by the recent short

survey of Beg et al. (2025) [1], which classifies the LLM-based literature in detail. It is further complemented with controlled-language frontends that the survey only briefly covers. Table 1 summarises the systems discussed in this section.

Controlled-language frontends. The longest-lived line of work attacks the problem at the *input* side by constraining the natural language itself. Nowakowski et al. (2013) [9] developed the *Requirements Specification Language* and the *ReDSeeDS* platform, which couples a constrained-NL requirements notation with automated transformations into code. Ghosh et al. (2016) [5] present *ARSENAL*, an NLP-based framework that extracts formal requirement specifications from natural language with automatic verification. NASA’s *FRET*, introduced by Giannakopoulou et al. (2020) [6], lets engineers write requirements in *FRETISH*, a restricted English with unambiguous semantics that maps deterministically to temporal logic.

LLM-only pipelines. With the rise of large language models, a growing body of work delegates the full translation to the LLM itself. Nayak et al. (2022) [8] present *Req2Spec*, an NLP pipeline that maps natural-language requirements onto the pattern-based formal specifications consumed by HANFOR; on 222 BOSCH automotive requirements it correctly formalises 71%. Cosler et al. (2023) [2] propose *nl2spec*, which uses few-shot prompting of Codex (code-davinci-002) and Bloom-176B to translate unstructured natural language into Linear-time Temporal Logic (LTL), with iterative user refinement of subformulas. Fang et al. (2024) [3] introduce *AssertLLM*, a three-stage pipeline of customised GPT-4o models (the final stage retrieval-augmented) that generates SystemVerilog Assertions from hardware design specifications with 89% correctness.

Hybrid LLM + prover systems. The most recent strand pairs an LLM with a formal prover or solver that has the final say on correctness. Jiang et al. (2022) [7] introduce *Thor*, which couples a 700M-parameter decoder-only transformer with the Isabelle/HOL theorem prover via Sledgehammer-based premise selection (“Hammers”), raising proof success on PISA from 39% to 57%. Yang et al. (2023) [11] release *LeanDojo* and the retrieval-augmented prover *ReProver*, a ByT5 encoder-decoder paired with premise retrieval over the Lean mathematics library and benchmarked on 98 734 theorems. Quan et al. (2024) [10] propose *ExplanationRefiner*, a refinement loop between an LLM (GPT-4, GPT-3.5-turbo, Llama2-70b, Mixtral-8x7b, or Mistral-small) and the Isabelle/HOL prover that validates and

Table 1. State-of-the-art systems for natural-language to formal-specification translation discussed in this section, in chronological order within each category. Abbreviations: HITL = human-in-the-loop; NLP = natural language processing; MDD = model-driven development; SAL = Symbolic Analysis Laboratory; LTL = linear-time temporal logic; HANFOR = an industrial requirements-analysis tool that consumes pattern-based formal specifications; SVA = SystemVerilog Assertions; HOL = higher-order logic (as in the Isabelle/HOL theorem prover); SMT = satisfiability modulo theories.

Ref.	System	Year	Models	Target	HITL
<i>Controlled-language frontends</i>					
[9]	RSL/ReDSeeDS	2013	— (rule-based)	Code (MDD)	Yes
[5]	ARSENAL	2016	NLP pipeline	SAL/LTL	No
[6]	FRET	2020	— (rule-based)	LTL	Yes
<i>LLM-only pipelines</i>					
[8]	Req2Spec	2022	NLP pipeline	HANFOR	No
[2]	nl2spec	2023	Codex, Bloom-176B	LTL	Yes
[3]	AssertLLM	2024	GPT-4o	SVA	No
<i>Hybrid LLM + prover systems</i>					
[7]	Thor	2022	700M trans-former	Isabelle/HOL	No
[11]	LeanDojo	2023	ByT5 + retrieval	Lean	No
[10]	ExplanationRefiner	2024	GPT-4/3.5, Llama2-70b, Mixtral-8x7b, Mistral-small	Isabelle/HOL	No
[4]	SAT-LLM	2024	LLM + SMT solver	SMT	No

corrects natural-language explanations. Fazelnia et al. (2024) [4] build *SAT-LLM*, which combines an LLM with an SMT solver to detect conflicting requirements at F1 0.91.

[[Add the gap and what is our contribution, basically summarize the next section]]

3. Contributing by designing a “close-to-NL” formalism

While prior controlled-language frontends [6, 9, 5] and LLM-based translators [2] target a single fixed logic — typically LTL or a tool-specific pattern language — none of the systems surveyed in Section 2 position their formalism around predicate-level matchability between requirements and test cases for coverage-gap detection, nor as a logic-agnostic intermediate explicitly designed to be the translation target for LLMs. The formalism presented in this section is designed around precisely these two properties.

109 3.1 Formalism requirements and its design

110 The formalism satisfies the properties listed below.
111 Each property is paired with a concrete use case ex-
112 pressed in abstract placeholders (signals A, B ; events
113 e_A, e_B ; etc.) and a brief note on the construct the
114 formalism provides to address it.

115 **[[Fix ordering]]**

116 **[[Include a requirement and test case example,**
117 **so the reader can have a better understanding be-**
118 **fore reading the rest]]**

119 **Meta-level properties. Mechanical coverage-gap**
120 **detection.** Given a set of formalised requirements
121 and a set of formalised test cases, the formalism shall
122 enable a checker to determine which requirement con-
123 ditions are not exercised by any test case. *Solution:*
124 Requirements and test cases are formalised over the
125 same entities and the same predicates, so that the same
126 sub-formulas appear on both sides. A checker can
127 then evaluate, for each test case, which meaningful
128 combinations of sub-formulas inside a requirement's
129 formula are driven to true and which to false, and from
130 that compute the fraction of relevant combinations that
131 the test suite exercises.

132 **Logic-agnostic representation of system behaviour**
133 **in time.** The formalism shall not commit to a specific
134 underlying logic, so that a single requirement repre-
135 sentation can be retargeted to different verification
136 back-ends. *Solution:* A requirement is represented
137 as a composition of two kinds of building blocks: (i)
138 ordinary expressions and predicates that, when eval-
139 uated against a trace (the same kind of object a test
140 case produces), yield concrete values or truth values
141 at each point in time; and (ii) temporal predicates that
142 aggregate those point-in-time predicates into a single
143 truth value over the entire trace. The semantics of
144 each building block is defined directly in set-theoretic
145 terms over traces and time, independent of any tar-
146 get back-end. Every temporal predicate has a direct
147 LTL (or metric-LTL) equivalent and a corresponding
148 timed-automaton construction.

149 **Extensible domains.** Real-world requirements
150 draw on an open-ended range of kinds of things in a
151 system (signals, registers, channels, states, and so on)
152 and of events that can happen to them (a register being
153 written, a channel transmitting a value, a state being
154 entered), and it is unrealistic to fix a closed catalogue
155 up front that covers every domain the formalism may
156 ever be applied to. The formalism shall therefore keep
157 these catalogues extensible. *Example:* To support a
158 new entity kind C representing network packets, only
159 the relevant catalogue is widened; the rest of the for-
160 malism is untouched. *Solution:* The set of entity kinds,

the set of value types, the assignment of permitted 161
attributes to each entity kind, and the assignment of 162
permitted events (parameterised by the entity they hap- 163
pen to) to each entity kind are all parameters of the 164
formalism rather than fixed sets. Adding a new entry 165
to any of them requires only registering it in the rele- 166
vant catalogue; existing constructs continue to work 167
unchanged. 168

Time and values. Discrete and continuous time. 169
The formalism shall support both discrete-time and 170
continuous-time requirements within a single vocabu- 171
lary. *Example:* A discrete-time requirement says “at 172
every step, A is incremented by 3”; a continuous-time 173
requirement says “ e_B must occur within 2 seconds of 174
 e_A ”. *Solution:* The time set T has two flavours: $T_D =$ 175
 $\mathbb{N} \cup \{\infty\}$ for discrete time and $T_C = \mathbb{R}_{\geq 0} \cup \{\infty\}$ for 176
continuous time. Each requirement carries a *flavour* $\in$ 177
 $\{\text{Discrete}, \text{Continuous}\}$ field that fixes T for its scope. 178

Entities of multiple kinds carry values, and op- 179
erations modify them in test cases. The formalism 180
shall represent entities of different kinds (registers, 181
states, communication channels, computed signals, 182
etc.) and let each carry a value at any given time 183
point. It shall also provide a way for test cases to de- 184
scribe how those values change over time. *Example:* 185
Register A , state B , and channel C each have a value 186
at every relevant time point; a test case may, for in- 187
stance, write a new value into A , transition B to a new 188
state, or transmit a value through C . *Solution:* The 189
formalism defines an extensible catalogue of entity 190
kinds (currently signal, storage, event, event-trigger, 191
channel, state, value, abstract, and set), and a valua- 192
tion maps each entity to its value at each time point of 193
a trace. Modifications of those values are expressed 194
through *operations* — an extensible catalogue of ac- 195
tions, currently including *calculate*, *write*, *read*, *fire*, 196
set_state, *transmit*, *receive*, *insert*, *remove*, and *clear* 197
— that a test case fires at specific time points to drive 198
the trace. Each entity kind has its own subset of appli- 199
cable operations (e.g. *write* applies to storage, *fire* to 200
event-triggers, *transmit* to channels). 201

Static entity-attribute access. The formalism 202
shall expose static attributes of an entity (independent 203
of trace and time). *Example:* The address of register 204
 A is the constant `0xAA000018` and can be referenced 205
as such. *Solution:* Static attributes are stored in an en- 206
tity's modifier map $\mu_e : \text{ModKey} \rightarrow \text{ModVal}$; accessors 207
such as $\text{Addr}(e) = \mu_e(\text{address})$ retrieve them without 208
reference to ρ or t . 209

Reference to an entity's value at another time 210
point. The formalism shall allow a constraint to refer 211
to the value an entity had (or will have) at an ear- 212

lier or later time point, enabling recurrences, toggles, and look-ahead conditions. *Example*: “ $A(\text{Now}) = A(\text{Now} - 1) + 3$ ”; “ B equals the negation of B ’s previous value”; or “ C at the next step equals the current value of D ”. *Solution*: $\text{Val}_\rho(e, t)$ returns the value of e at any time t , past or future; for discrete time, *Prev*, *Next*, and *RelTime* shift the time argument backward or forward; $\text{ValBefore}_\rho(e, t)$ exposes the left limit at t .

Arithmetic on values. The formalism shall support standard arithmetic on numeric values so that constraints can express computations over entity values. *Example*: “ $A = B + 3$ ”; or “ $C = 1.2 \cdot D$ ”. *Solution*: The standard arithmetic operators $+$, $-$, $\times$, $/$ are defined on numeric values and can appear inside any expression.

Time arithmetic and intervals. The formalism shall support arithmetic on time points and the explicit construction of intervals with controlled endpoint openness. *Example*: “the duration elapsed between e_A and e_B ”; “the half-open interval $(t_1, t_2]$ ”. *Solution*: *Diff* returns the duration between two time points; the four interval constructors *MkIntervalCC*, *MkIntervalOC*, *MkIntervalCO*, *MkIntervalOO* build intervals with the chosen open/closed endpoints.

Events and triggers. Event occurrence at a given time point. The formalism shall express that an event occurs at a particular time point. *Example*: “ e_A is happening at t .” *Solution*: The predicate *Happening*(e, t), where e is an entity of type *Event*.

Past-event semantics. The formalism shall express that an event has occurred at some point up to the current time. *Example*: “ e_A has happened at least once.” *Solution*: The predicate *HasHappened* $_\rho(e, t)$.

Reference to event-occurrence times. The formalism shall expose the time of a specific past or future occurrence of an event. *Example*: “the time of the most recent occurrence of e_A .” *Solution*: *LastOcc* $_\rho(ev, t, n)$ returns the interval spanning the n most recent occurrences of ev up to t ; *FirstOcc* $_\rho$ does the same in the forward direction.

Counting event occurrences within an interval. The formalism shall count how many times an event occurred within a given interval, with control over whether interval endpoints are included. *Example*: “the number of occurrences of e_A in the half-open interval $(t_1, t_2]$.” *Solution*: *EvtOccCount* $_\rho(ev, i)$ returns the count.

Conditional and logical structure. Conjunction, disjunction, and negation of predicates. The formalism shall combine predicates of any kind (event happenings, value comparisons, occurrence counts,

historical values) into a single conjunctive or disjunctive guard, and shall negate any predicate. *Example*: “ e_A and $B > 100$ and e_C has occurred at least twice”; “ e_A or e_B or e_C ”; or “ A is not equal to B ”. *Solution*: *AllOf*, *AnyOf*, and *Not* form the conjunction, disjunction, and negation of predicates drawn from the shared predicate space.

Value-conditional branches. The formalism shall support constraints that branch on the current value of an entity. *Example*: “If A is true then B equals X ; otherwise B equals Y .” *Solution*: A value-guarded branch is expressed as the conjunction of two implications using *AllOf*, *Implies*, and the comparison predicate *Cmp*, for example: *AllOf*($\{\text{Implies}(\text{Cmp}(A, =, \text{true}), \text{Cmp}(B, =, X)), \text{Implies}(\text{Cmp}(A, =, \text{false}), \text{Cmp}(B, =, Y))\}$).

Sets, set membership, and set quantification.

The formalism shall represent finite sets of values, test whether a value belongs to a set, quantify over a finite set of entities, and reason about properties of the subset that satisfies a predicate. *Example*: “the value v belongs to set S ”; “for every entity in $\{A, B, C, D\}$, predicate P holds”; or “at most two entities in $\{A, B, C, D\}$ have a value greater than 10.” *Solution*: The Set entity kind carries set values; *Contains* tests membership; the quantifiers *ForAll*(S, P) and *Exists*(S, P) range over a finite set, with *Filter* and *Size* supporting cardinality reasoning over the subset satisfying a predicate.

Temporal patterns. Initial-value constraints. The formalism shall be able to constrain the value of an entity at the initial time point. *Example*: “Signal A has initial value 0.” *Solution*: The temporal construct *Initial*(ψ) $\Leftrightarrow \psi(\rho, 0)$ asserts a point-in-time predicate at time 0.

Existence, global constraints, and trace-level composition. The formalism shall express both “at some point in the trace” and “at every time point in the trace”, and shall let multiple such trace-level constraints be combined into a single requirement. *Example*: “Eventually A equals 5”; “ A is always non-negative”; or “Always ψ_1 and eventually ψ_2 ”. *Solution*: *Eventually*(ψ) and *Always*(ψ) produce trace-level truth values; multiple trace predicates compose via the standard logical operators ($\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$) applied at the trace level.

Sequencing and time-bounded causation. The formalism shall require multiple actions to occur in a specified order, and shall require an effect to occur within a bounded duration of a cause. *Example*: “ e_A then e_B then e_C ”; “if e_A occurs, e_B must occur within 2 seconds”. *Solution*: *Sequence*($\psi_1, \dots, \psi_n$) and *CausesWithin*($\psi_{\text{cond}}, \psi_{\text{effect}}, d$), respectively; sim-

ilarly, $Causes(\psi_{cond}, \psi_{effect})$ for unbounded causation, $Precedes(\psi_1, \psi_2)$ for ordering, $Excludes(\psi_1, \psi_2)$ for mutual exclusion, and $Immediately(\psi_1, \psi_2)$ for next-step succession.

Sliding temporal windows. The formalism shall reason about a window of fixed length ending at the current time. *Example:* “ e_A occurred at least twice in the last 10 seconds.” *Solution:* An interval constructor applied to *Now* (e.g. $MkIntervalOO(Now - d, Now)$) produces the window, which can then be passed to $EvtOccCount_p$ to count event occurrences, to $MaxVal_p/MinVal_p$ to bound a signal’s range, or to interval predicates such as $IntervalIncludes/IntervalExcludes$.

[[Preferably without human in the loop]]

3.2 Formalism design decisions

[[Might already be covered by previous section]]

3.3 Formalism definition

[[Add an appendix]] [[Add appendix with examples]]

3.4 Known limitations and possible future extensions

[[Although this work claims that the formalism should be close to english and is close in the sense that it uses english words, the words themselves might not appear in a similar order than they would in english. This might be a limitation.]]

[[Add limitations from formalism document]]

3.5 Compatibility with well-known formal specification methods

3.6 Requirement and test case examples

3.7 Future work

4. Conclusions

Acknowledgements

I would like to thank my supervisor Ing. Aleš Smrčka Ph.D. for his help.

References

- [1] BEG, A., O’DONOGHUE, D. and MONAHAN, R. *A Short Survey on Formalising Software Requirements with Large Language Models*. 2025. Available at: <https://arxiv.org/abs/2506.11874>.
- [2] COSLER, M., HAHN, C., MENDOZA, D., SCHMITT, F. and TRIPPEL, C. *NI2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language*

Models. 2023. Available at: <https://arxiv.org/abs/2303.04864>.

- [3] FANG, W., LI, M., LI, M., YAN, Z., LIU, S. et al. AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs. In: *2024 IEEE LLM Aided Design Workshop (LAD)*. 2024, p. 1–1.
- [4] FAZELNIA, M., MIRAKHORLI, M. and BAGHERI, H. Translation Titans, Reasoning Challenges: Satisfiability-Aided Language Models for Detecting Conflicting Requirements. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE ’24)*. New York, NY, USA: Association for Computing Machinery, 2024, p. 2294–2298.
- [5] GHOSH, S., ELENIOUS, D., LI, W., LINCOLN, P., SHANKAR, N. et al. ARSENAL: Automatic Requirements Specification Extraction from Natural Language. In: RAYADURGAM, S. and TKACHUK, O., ed. *NASA Formal Methods*. Cham: Springer International Publishing, 2016, p. 41–46.
- [6] GIANNAKOPOULOU, D., MAVRIDOU, A., RHEIN, J., PRESSBURGER, T., SCHUMANN, J. et al. Formal Requirements Elicitation with FRET. In: *Joint Proceedings of REFSQ-2020 Workshops, Doctoral Symposium, Live Studies Track, and Poster Track*. Pisa, Italy: [b.n.], 2020.
- [7] JIANG, A. Q., LI, W., TWORKOWSKI, S., CZECHOWSKI, K., ODRZYGÓŹDŹ, T. et al. Thor: Wielding Hammers to Integrate Language Models and Automated Theorem Provers. In: *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2022, p. 8360–8373.
- [8] NAYAK, A., TIMMAPATHINI, H. P., MURALI, V., PONNALAGU, K., VENKOPARAO, V. G. et al. Req2Spec: Transforming Software Requirements into Formal Specifications Using Natural Language Processing. In: *Requirements Engineering: Foundation for Software Quality (REFSQ 2022)*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 87–95.
- [9] NOWAKOWSKI, W., ŚMIAŁEK, M., AMBROZIEWICZ, A. and STRASZAK, T. Requirements-Level Language and Tools for Capturing Software System Essence. *Computer Science and Information Systems*. 2013, vol. 10, no. 4, p. 1499–1524.

- 411 [10] QUAN, X., VALENTINO, M., DENNIS, L. A.
412 and FREITAS, A. *Verification and Refinement*
413 *of Natural Language Explanations through LLM-*
414 *Symbolic Theorem Proving*. 2024. Available at:
415 <https://arxiv.org/abs/2405.01379>.
- 416 [11] YANG, K., SWOPE, A., GU, A., CHALAMALA,
417 R., SONG, P. et al. LeanDojo: Theorem Prov-
418 ing with Retrieval-Augmented Language Models.
419 In: *Advances in Neural Information Processing*
420 *Systems*. Curran Associates, Inc., 2023, p. 21573–
421 21612.